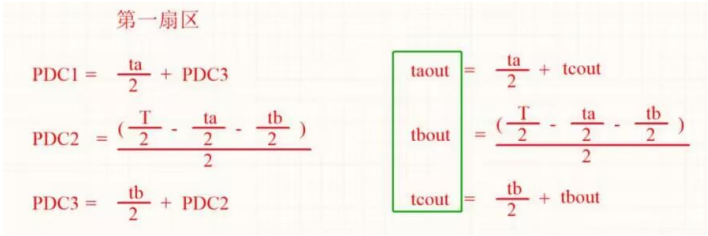
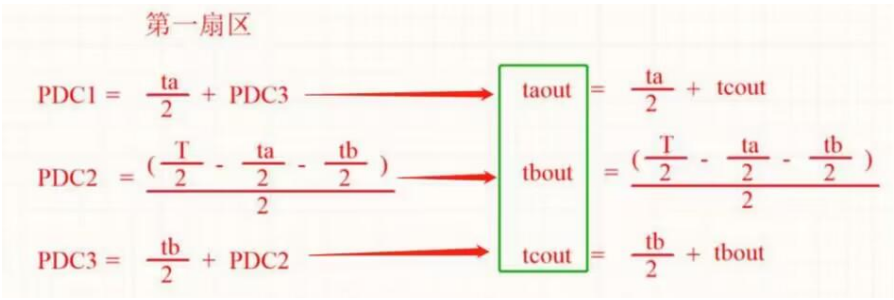
37-SVPWM 8

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。

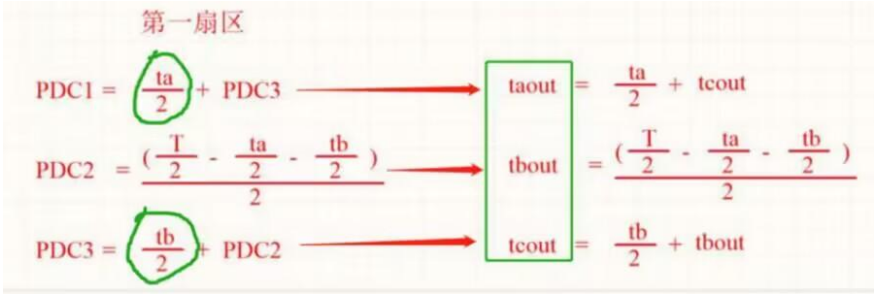
上次课最后，我们想对六个扇区对应的 PDCx 的表达式再进一步去做一些变换，目的呢，是想要我们在编写软件的时候更为方便。



例如，这个第一扇区，有这样的关系。这个关系的得出，是利用了每个扇区的第一作用矢量　跟第二作用矢量。这里的第一跟第二作用矢量，是根据矢量的排列顺序来划分的　把第一作用矢量的作用时间定义为 ta，第二作用矢量的时间定义为 tb。那么，每个扇区都有对应的第一作用矢量跟第二作用矢量。所以，咱们可以提炼出一个全局的公式，这个公式呢，适用于六个扇区，仅是不同扇区在最后赋值给 PDCx 的时候，对应的顺序会不同，这样的话，我们就定义了 taout，tbout，tcout 三个量。

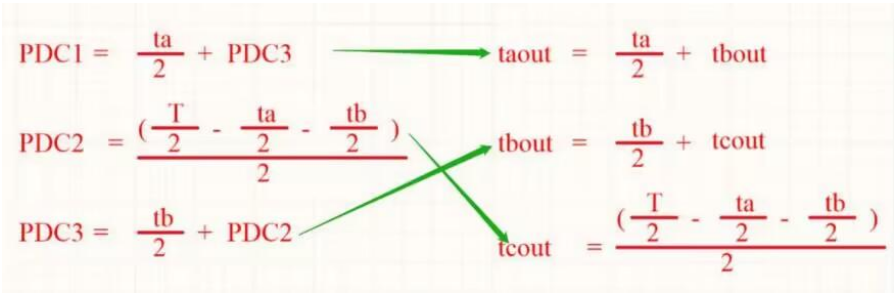


在第一扇区呢，它们的对应关系是这样的。为了更好的对应，我们做这样的变化。



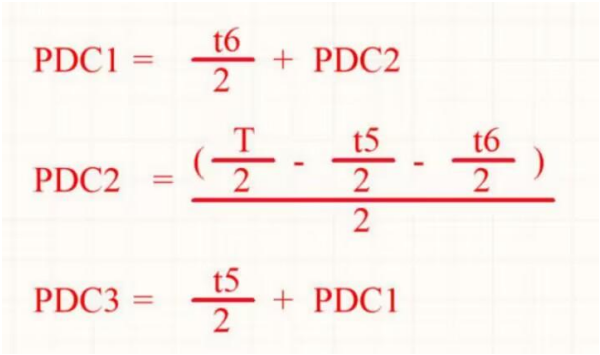
大家看，左边的表达式中，PDC1 利用了第一矢量作用时间，PDC3 利用了第二矢量作用时间，而 PDC2 是对应零矢量的作用时间。所以，我们可以定义，第一矢量作用的，用 taout 来代替，第二作用矢量作用的，用 tbout 来代替，零矢量的，用 tcout 来代替。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



转换之后，就这样来表示，这是第一扇区的情况，这里，大家可以理解吧？

好，然后我们继续来看。

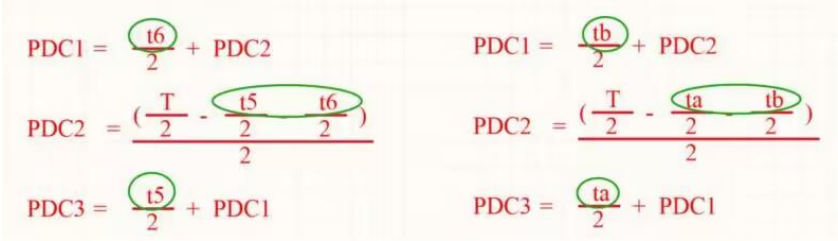


第二扇区，对应的 PDCx 表达式是这样的。

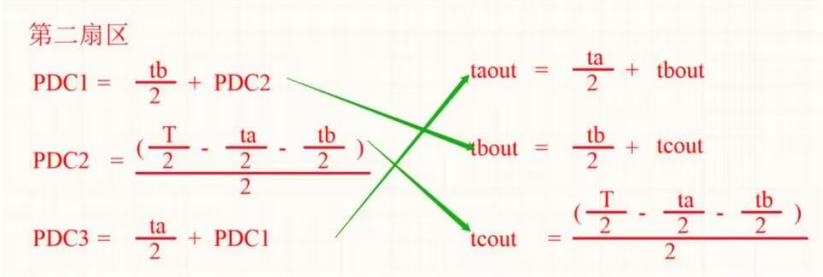
在第二扇区



同时呢，这个扇区的矢量作用顺序是这样的，所以，V5 就是第一作用矢量，V6 就是第二作用矢量。因此，t5 就可以用 ta 来代替，t6 可以用 tb 来代替。



代替之后，就是这样的。



所以，第二扇区，可以这样表示。 换句话说，我们可以把右边的三个式子看成是公共的式子，不同扇区的 PDCx 就是在这三个式子中去选择组合。第二扇区的话，可以对应：

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

PDC1 = tbout PDC2 = tcout PDC3 = taout，这样做的目的，只要我们把右边的三个公共式子求取出来，那么 PDCx 也就对应扇区组合一下，就得到了。

我们再来看一个第三扇区，再总结一下规律。



第三扇区，矢量作用顺序是这样的。那么，第一矢量就对应 V5，第二矢量就对应 V4。

$$PDC1 = \frac{(\frac{T}{2} - \frac{t5}{2} - \frac{t4}{2})}{2}$$
$$PDC2 = \frac{t4}{2} + PDC1$$
$$PDC3 = \frac{t5}{2} + PDC2$$

$$PDC1 = \frac{(\frac{T}{2} - \frac{ta}{2} - \frac{tb}{2})}{2}$$
$$PDC2 = \frac{tb}{2} + PDC1$$
$$PDC3 = \frac{ta}{2} + PDC2$$

所以，第三扇区就可以变为右边这样。

$$PDC1 = \frac{(\frac{T}{2} - \frac{ta}{2} - \frac{tb}{2})}{2}$$
$$PDC2 = \frac{tb}{2} + PDC1$$
$$PDC3 = \frac{ta}{2} + PDC2$$

$$taout = \frac{ta}{2} + tbout$$
$$tbout = \frac{tb}{2} + tcout$$
$$tcout = \frac{(\frac{T}{2} - \frac{ta}{2} - \frac{tb}{2})}{2}$$

再跟公共的式子对比，就可以得到，第三扇区的 PDCx ：
PDC1 = tcout PDC2 = tbout PDC3 = taout
那么，最后总结一下，会得到下面的表。

$$taout = \frac{ta}{2} + tbout$$
$$tbout = \frac{tb}{2} + tcout$$
$$tcout = \frac{(\frac{T}{2} - \frac{ta}{2} - \frac{tb}{2})}{2}$$

第六扇区	第五扇区	第四扇区	第三扇区	第二扇区	第一扇区
PDC1 = taout	PDC1 = tbout	PDC1 = tcout	PDC1 = tcout	PDC1 = tbout	PDC1 = taout
PDC2 = tbout	PDC2 = taout	PDC2 = taout	PDC2 = tbout	PDC2 = tcout	PDC2 = tcout
PDC3 = tcout	PDC3 = tcout	PDC3 = tbout	PDC3 = taout	PDC3 = taout	PDC3 = tbout

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

这里想给大家表达的意思就是，六个扇区的 PDCx 寄存器不需要单独去求。而是可以写一个带形参的函数，这个函数的输入是 ta,tb, 输出是 taout tbout tcout。后续要赋值给 PDCx 寄存器的时候，我们就直接根据扇区的判断，直接用求解出来 taout tbout tcout 三个值赋值，不然每个扇区都求解一次，会显得复杂。

好，这样的话，SVPWM 控制算法的原理，我们就分析了一遍。

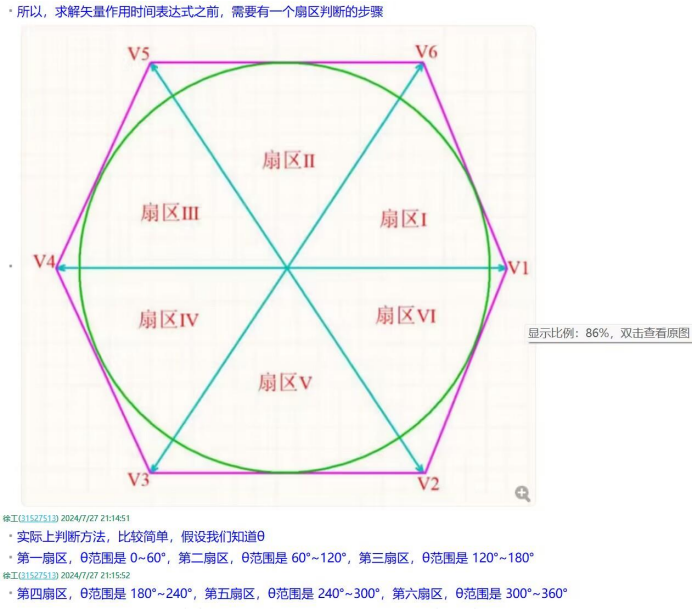
大家看看有没有疑问？后面我们就要开始准备编程了。

问：是先讲 svpwm 的编程部份再讲后面的算法内容吗？

答：是的。

问：扇区判断是在编程部分讲解吗？

答：

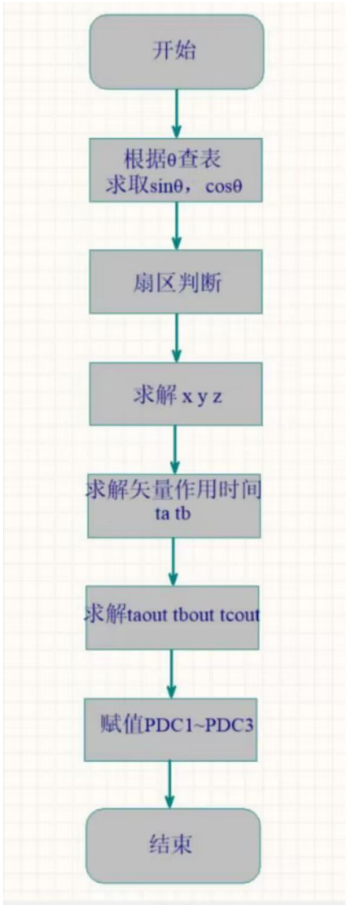


问：可以总结一下吗？

答：SVPWM 原理主要围绕三个 PWM 发生器的比较值计算展开，模块输入变量是 Us，输出就是 PDC1,PDC2,PDC3,求解矢量作用时间是重要的中间过程。

那么，根据我们之前的分析，其实我们可以绘制出一幅流程图，这样的话，会方便我们来编程。

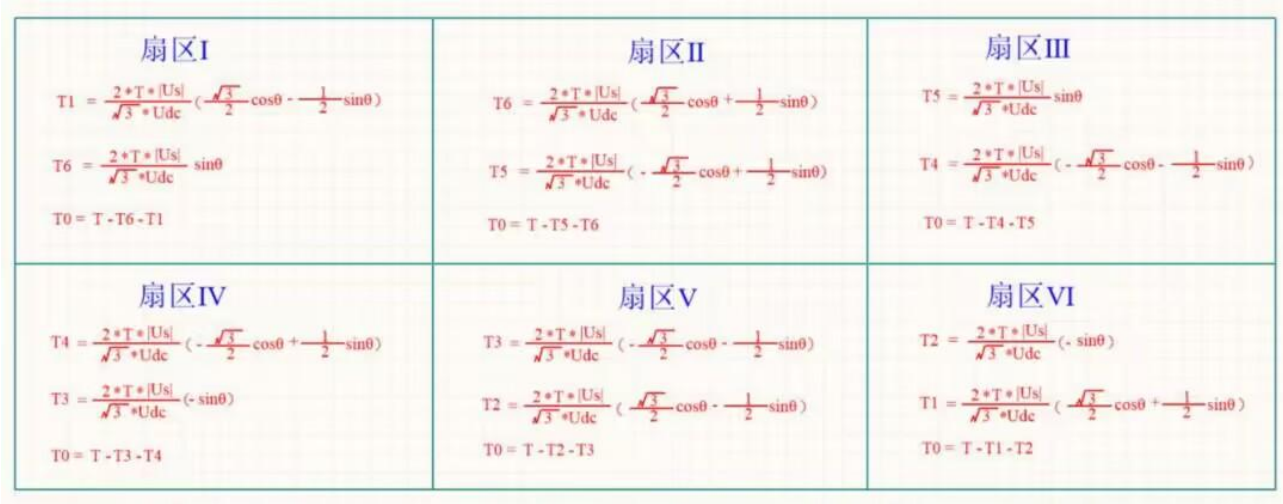
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



流程画出来，其实就是这样的。



整个流程图呢，其实就可以等效于这个 SVPWM 模块。模块有两个输入，三个输出。输入的 θ，对应的是电压合成矢量跟 0° 位置的夹角，|Us|对应的是电压合成矢量的模长。



本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

根据这个表，我们可以知道，已知 θ 可以获取 sin θ 跟 cos θ 。然后，再结合|Us|来求解矢量作用时间。而不同扇区的矢量作用时间表达式不同，但是它们有一些公因式。

$$x = \sin\theta$$
$$y = \frac{1}{2}\sin\theta + \frac{\sqrt{3}}{2}\cos\theta$$
$$z = \frac{1}{2}\sin\theta - \frac{\sqrt{3}}{2}\cos\theta$$

那么，经过提炼，得出这三个公因式。这样的话，就得出了用三个公因式表示的矢量作用时间的表达式。

扇区I	扇区II	扇区III
$T1 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(-z)$ $T6 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}x$ $T0 = T-T6-T1$	$T6 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(y)$ $T5 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(z)$ $T0 = T-T5-T6$	$T5 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(x)$ $T4 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(-y)$ $T0 = T-T4-T5$
扇区IV	**扇区V**	**扇区VI**
$T4 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(z)$ $T3 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(-x)$ $T0 = T-T3-T4$	$T3 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(-y)$ $T2 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(-z)$ $T0 = T-T2-T3$	$T2 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(-x)$ $T1 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(y)$ $T0 = T-T1-T2$

所以，输入角度 θ 跟|Us|是为了求解矢量作用时间。

矢量作用时间求取出来，是为了要得到对应的 PDCx。只不过在得到 PDCx 值之前，中间需要有一些转换，转换的目的，是为了让算法编程实现较简便。

问：这个中间的转换没有怎么理解？
答：就是指前面的 taout, tbout, tcout。

问：原理不是很明白？
答：先要搞清楚第一矢量, 第二矢量，还有 ta, tb 的定义，再看前面的推导过程。自己推一遍，看看有没有堵点，不能光看，一定要自己动手推一下，就会懂了，推不通的时候再看晚课资料。

问：SVPWM 是不是可以单独作为一个模块实现无刷电机驱动？
答：可以的。

问：是让角度自己自增，然后 us 自己给吗？
答：这种是强拖控制，也可以通过传感器位置反馈来给定 Us。

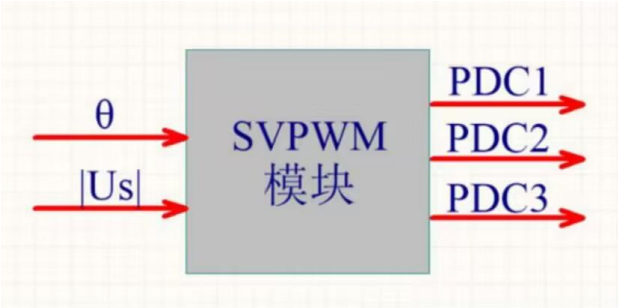
问：马鞍波行程过程介绍下呢？
答：这里 SVPWM 的调制方式，线电压是马鞍波，相电压是正弦波。不同于 SPWM 调制波，线电压是正弦波，相电压也是正弦波。主要原因是 SVPWM 中叠加了三次谐波。

问：按照逆时针第一矢量是 V1，第而矢量时 V2，那么顺时针呢？

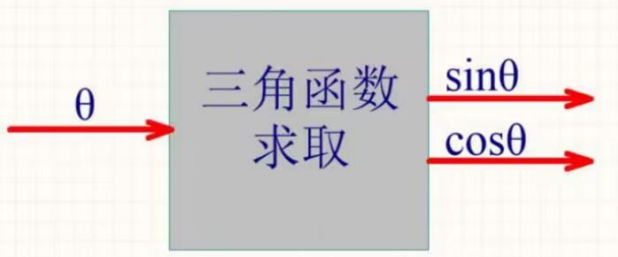
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

答：不变，第一矢量，第二矢量是根据矢量的作用顺序来定义的。和角度的正方向定义有关。

好，那我们就继续往下讲了。



这个模块中，|Us|我们会人为来给定，所以，这里它就相当于是已知的量。 目前阶段我们先考虑开环 SVPWM 如何实现，后面再讨论在 FOC 中如何应用。先考虑开环 SVPWM,也就是 Us 给定，如何产生 SVPWM。而这个 θ，输入到模块中，是要得到 sin θ 跟 cos θ 的。倘若 θ 已知，那么要求解 sin θ，cos θ 的话，要怎么求解呢？



实际上，这里又可以看成是一个子模块，这个子模块的输入是 θ，输出是 sin θ 及 cos θ 也就是说，输入一个角度，要得到角度对应的正弦值跟余弦值。换句话说，就是要求解角度对应的三角函数值。怎样知道这个三角函数值呢？

问：线电压应该也是正弦波，端电压是马鞍波？
答：SVPWM 线电压是马鞍波，端电压是马鞍波。

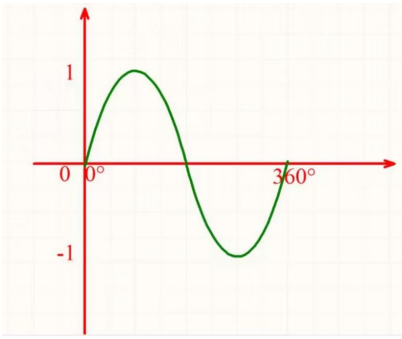
是的，前面同学提到有使用查表，那么，使用什么方法，要看我们使用芯片的能力 能力不足，没有浮点运算能力，那么就无法使用浮点运算库。也就是说，在不具备浮点运算单元的单片机中，我们是不能够直接去计算浮点运算的。而我们使用的单片机，确实没有浮点运算单元。所以，我们这里要计算三角函数的话，要利用查表法。也就是说，我们是事先把某些角度对应的正弦值先求解出来。求解出来之后呢，我们做成一个表，然后在编程的时候，就根据角度去查表。查出来的值，就是该角度对应的正弦值。所以啊，我们就重点来讲解这个查表法的实现。

这个查表法的话，其实分为两个步骤：

1. 正弦表的制作；
2. 编写查表算法。

我们先来看下正弦表是怎么制作的？

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



大家看下，这是一个正弦波周期，实际上，做表的方式有很多种。第一种，就是做一个 360° 的表，也就是说，这个表包含了，0°~360° 这个角度范围内的正弦值。这种表的话，我们也称为 360° 的表。第二种，做一个 180° 的表。然后，其它角度的话，根据波形的关系，可以求解得到。第三种，做一个 90° 的表，其它角度的话，根据波形的关系，同样可以求解得到。实际上，如果这三种情况，制作的正弦表的大小是一样的话，那么其实精度比较高的是 90° 这种情况。

假定我们要做的表的大小是 128 个数。那么，360 的表，就是在 0~360° 这个角度范围内，取 128 个角度，来计算对应的正弦值，组成一张表。而 180° 的表，就是在 0~180° 这个角度范围内，取 128 个角度，计算对应的正弦值，组成一张表。90° ，也是同理。所以，这样对比下来，肯定是 90° 这种情况，取 128 个点，对应的表的精度会更高。

那么，虽然这种方法精度高，但是，在 90° 外的其它角度对应的值，就要通过波形关系来转换得到。也就是说，算法中会涉及到一些判断处理步骤。这种方法的查表实现呢，留给大家自行练习。我们课上呢，主要讲解 360° 的表这种方式。

那么，既然刚才提到这种方式精度相对较差，为什么还要使用这种方法呢？

首先呢，它查表比较快速方便，整个 0°~360° 输入的角度，都可以直接查表得到对应的值。不需要做什么转换，其次，如果我们能够找到一些方法，提升这种方法的精度，它也不是不可以使用的。假设我们就制作一张 128 个正弦值的表，那么，我们可能就会碰到输入的角度不在这 128 个角度当中的情况，此时，如果我们直接用查表的结果作为输入角度的正弦值，那么就会出现误差。

问：DSP 有没有带浮点功能的片子呢？

答：肯定有啊，价格会贵一些。

问：90 度这种方式，精度高吗？

答：90 度方式相比 180 度和 360 度方式，精度高。

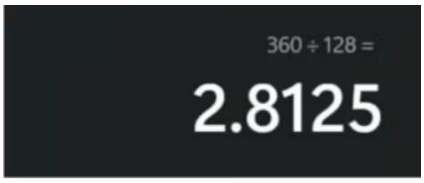
那么为了减小误差，我们会引入一个线性插值的方法。也就是说，当碰到输入角度在表中找不到对应的正弦值的时候，我们利用线性插值的方式，把这个角度对应的正弦值给计算出来。这样，我们使用 360° 的正弦表的时候，也可以得到精度相对较高的正弦值，也就是，这种线性插值方法，弥补了正弦表点数少带来的精度差的情况。

使用点数少，是因为有的低端单片机，内存空间也相对比较小 ，性能也不是很高，内存也不是很大，这样我们就不能把表做得很大。所以，也就想到了查表+线性插值的方式。如果说，大家使用的单片机内存很大，那么大家也可以把表做大一些，再结合线性插值，精度就会更高些。好，这是做表前，稍微给大家提一下，为什么要这样做表。下面我们就来看下，正弦表的制作。这个呢，可以借助 EXCEL 来完成。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

	度数	角度增量	sin值
2	0		0
3	2.8125	2.8125	0.049067674
4	5.625	2.8125	0.09801714
5	8.4375	2.8125	0.146730474
6	11.25	2.8125	0.195090322
7	14.0625	2.8125	0.24298018
8	16.875	2.8125	0.290284677
9	19.6875	2.8125	0.336889853
10	22.5	2.8125	0.382683432
11	25.3125	2.8125	0.427555093
12	28.125	2.8125	0.471396737
13	30.9375	2.8125	0.514102744
14	33.75	2.8125	0.555570233
15	36.5625	2.8125	0.595699304
16	39.375	2.8125	0.634393284
17	42.1875	2.8125	0.671558955
18	45	2.8125	0.707106781

大家来看下，首先，我们要制作 128 个点的正弦表，那么，我们就要取 128 个角度。而这 128 个角度是在 0~360° 范围内取的，所以，两个相邻角度点的角度间隔，我们可以求解出来。



求解方法，就是用 360°/128，也就可以得到角度增量是 2.8125°，所以，这 128 个角度，就从 0° 开始，按照 2.8125° 的角度增量去递增。然后，使用 ECXEL 拉一下，就可以得到上表中的第一列角度值。

问：咱们这个全部课程的代码占比到 80%了么？

答：远远没有，瓶颈不在存储容量，而在执行时间。

上幅图是 128 个角度的部分角度截图。然后，表中的第二列数据是为了要计算下一个角度值，写的一个角度增量。接着呢，角度写出来了，就可以利用 excel 中的公式，来把角度对应的正弦值给计算出来。表中的第三列数据，就是第一列数据对应的正弦值。这样的话，这个表就制作好了。

我们接着来看啊，从表中的数据来看，这些数据都是浮点数。而我们单片机没有浮点运算单元。所以，处理浮点数的话，会比较困难。因此啊，碰到这种情况，我们需要把浮点数转换一下。可以把小数放大 N 倍，这是一种方法，也就是把小数放大 N 倍，变为整数。整数的运算的话，几乎所有的单片机都是可以实现的。

那么要放大多少倍呢？小数点是在浮动变化的。小数点后面的数越少，放大这个倍数对应的结果就越大。小数点后面数越多，可能放大这个倍数，又不够。所以要依据数据来灵活调整放大的倍数。浮点转整数，还会碰到精度丢失的情况，这是必然的。但是，虽然丢失了精度，也不是说做项目的时候就不能这样用。就是说，控制电机的效果比浮点运算的效果稍

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

差一些，但是并不意味着不能使用。很多项目，精度要求也没有那么的高。并不一定就要使用浮点计算。而且，带浮点运算单元的单片机，价格会比较高。所以，在做项目的时候，又要考虑成本要求。所以啊，基于这样的原因，在项目对成本要求高的场合，我们会选择一些低端单片机。

今天就讲到这里，大家有没有问题？

问：怎么看表占多少内存？

答：表不是占内存，是占 Flash 空间，多大只要数一数数据个数，或者通过仿真器看一下就知道了。

问：这个编码会逐句讲解一遍么？

答：会的。

问：我计算 Park 的那个角度，是 d 轴和 0 度 alpha 的角度，和你这里 SVPWM 的角度 θ 是一个角度吗？

答：不是，SVPWM 中的角度是指合成矢量 Us 与零度夹角。

问：与比较寄存器比较输出逻辑变了，比如原来大于载波值输出高，现在反过来怎么调整？

答：单片机一般都有反向输出的寄存器配置，更改这个配置就可以了。

问：这个小数放大 n 倍与 q15 哪个更好？

答：本质上是一样的，Q15 就是放大了 32678 倍。

问：现在讲的这些和 Clarke，Park，反 Park，反 Clarke 有什么关系？

答：没有关系。

问：我想人为让它反转，怎么调整呢？

答：正转角度递增，反转就是角度递减。

谢谢大家，晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。